

Mid-Project Report: A Multimodal RAG System for Astronomy

Team Knowlagents

Gargi Shroff (2022114009)
Ameya Rathod (2022111021)

October 25, 2025

Abstract

This report details the mid-project status of a specialized Retrieval-Augmented Generation (RAG) system for the domain of Astronomy. We have successfully implemented an end-to-end data ingestion, processing, and indexing pipeline capable of handling heterogeneous document formats (PDF, DOCX, PPTX, MD, TXT) and web-scraped content. A key feature of our approach is a multimodal embedding strategy, which allows us to index both text chunks and extracted images into a shared vector space using Elasticsearch. This pipeline serves as the foundation for our AI Subject Matter Expert (SME), the "Cosmic Compass," designed to provide synthesized, context-aware, and visually-supported answers to complex astronomical queries.

Contents

1	AI Agent Definition: The Cosmic Compass	2
1.1	Domain	2
1.2	Agent Name	2
1.3	Core Role	2
1.4	Key Capabilities	2
1.5	Limitations	2
2	Document Collection & Organization	2
2.1	Corpus Collection and Justification	3
2.2	Heterogeneous Format Support	3
2.3	Metadata and Organization	3
3	Preprocessing & Chunking	4
3.1	Preprocessing Pipeline	4
3.2	Chunking Strategy: Content-Aware Hierarchical	4
4	Embedding & Indexing	5
4.1	Embedding Model: Multimodal (CLIP)	5
4.2	Vector Store: Elasticsearch	5
4.3	Indexing Pipeline	5
5	Conclusion & Next Steps	6

1 AI Agent Definition: The Cosmic Compass

As per the project requirements, we first define the specialized domain and role of our AI agent.

1.1 Domain

K-12 Education: Astronomy and Cosmology.

1.2 Agent Name

The Cosmic Compass

1.3 Core Role

The Cosmic Compass is an AI Subject Matter Expert (SME) designed to assist researchers, students, and enthusiasts in navigating and synthesizing complex astronomical knowledge. Unlike a general-purpose search engine, its primary function is not just to find facts, but to **connect concepts** across a curated, authoritative corpus and present them with relevant textual and visual context.

1.4 Key Capabilities

- **Multimodal Conceptual Search:** The agent leverages a multimodal embedding model (CLIP) to understand queries in the context of both text and images. A user can ask, "Show me what a planetary nebula looks like," and receive both the technical definition and relevant diagrams or photographs from the knowledge base.
- **Adaptive Learning Support:** For student users, the agent can generate adaptive quizzes based on specific topics (e.g., "Quiz me on the Hertzsprung-Russell diagram"). It can also provide step-by-step explanations for complex processes, citing its sources.
- **Research "Frontier" Identification:** By analyzing the ingested literature (e.g., textbooks vs. recent articles), the agent can help identify and summarize open questions or "frontier topics" in a specific sub-field.

1.5 Limitations

- **Static Knowledge Base:** The agent's knowledge is strictly limited to the ingested corpus. It cannot access real-time astronomical data (e.g., "Where is Mars in the sky tonight?") or comment on news published after the last data ingestion.
- **Domain-Specific:** Its expertise is deep but narrow. It cannot answer questions outside the domain of astronomy and cosmology.

2 Document Collection & Organization

This section details the fulfillment of the data collection requirements.

2.1 Corpus Collection and Justification

We have curated a specialized corpus for the Astronomy domain, stored in a root-level `docs/` directory. Our collection strategy, as per the guidelines, focuses on foundational textbooks and supplementary authoritative content.

- **Authoritative Textbooks (Core):** We have ingested three primary textbooks to serve as the foundational knowledge base:
 - *Astronomy for Mere Mortals*
 - *Astronomy* (OpenStax edition)
 - *Introduction to Astronomy and Cosmology*
- **Supplementary Content (Web-Scraped):** To augment the textbooks with more focused, recent, and accessible content, we implemented a web scraper (`scrape_articles.py`). This script targets `science.nasa.gov` and can be used to collect articles on topics like black holes, exoplanets, and stars if required.

Justification: This combination provides a robust and balanced corpus. The textbooks offer deep, structured, and comprehensive knowledge, while the NASA articles provide supplementary, high-quality content on specific subjects.

2.2 Heterogeneous Format Support

The ingestion pipeline (`file_ingest.py`) is designed to automatically detect and process a variety of file types, fulfilling the heterogeneous format requirement.

- **PDF (.pdf):** Processed using the `fitz` (PyMuPDF) library. This allows for the extraction of both text and embedded images, which are saved separately.
- **DOCX (.docx):** Processed using `docx2txt`, which effectively extracts text and linked images.
- **PPTX (.pptx):** Processed using `python-pptx`. Text is extracted from all shapes on all slides, and images are extracted and saved.
- **Markdown (.md):** A custom parser identifies local image links (e.g., ``), copies the images to a central `images/` directory, and replaces the link with a placeholder.
- **Text (.txt):** Read directly.

2.3 Metadata and Organization

For every source document processed, the `file_ingest.py` script generates a detailed JSON manifest file, which is saved in the `METADATA_DIR`. This manifest serves as the "source of truth" for our indexing pipeline.

Each manifest contains:

- `doc_id`: A unique identifier derived from the filename.
- `source_file`: The absolute path to the original document.

- **assets:** A dictionary containing lists of filenames for all associated files generated during processing, including:
 - **images:** A list of all images extracted from the document.
 - **child_chunks:** A list of all small (256-token) text chunk files.
 - **parent_chunks:** A list of all large (1024-token) text chunk files.

This metadata structure is essential for linking embeddings back to their source and for managing the parent-child relationships in the index.

3 Preprocessing & Chunking

3.1 Preprocessing Pipeline

All extracted text undergoes a basic cleaning process via the `preprocess_text` function. This involves:

1. **Lowercasing:** All text is converted to lowercase to ensure uniformity.
2. **Whitespace Normalization:** Multiple spaces, tabs, and newlines are collapsed into a single space.
3. **Tokenization:** While not an explicit preprocessing step in that function, tokenization is performed implicitly by the `tiktoken` library (using the `cl100k_base` encoding) to determine chunk sizes accurately.

3.2 Chunking Strategy: Content-Aware Hierarchical

We rejected a fixed-size chunking strategy as it often cuts sentences and paragraphs in half, destroying semantic meaning. Instead, we implemented a **content-aware hierarchical** strategy in the `chunk_text` function.

Justification: This approach provides the best of both worlds: semantic precision for search and broad context for generation. It directly supports the requirement for multiple granularities and hierarchical retrieval.

The process works as follows:

1. **Content-Aware Splitting:** The text is first split by newline characters (`'\n'`). This respects the document’s natural structure, treating paragraphs as the primary semantic units.
2. **Child Chunks (256 Tokens):** We iterate through these paragraphs to create small, semantically-dense "child chunks."
 - If a paragraph is smaller than 256 tokens, it becomes a single child chunk.
 - If a paragraph is **larger** than 256 tokens, it is split by sentences. Sentences are then re-combined until they form a chunk just under the 256-token limit.

These child chunks are ideal for embedding and vector search, as they contain focused, specific information.

3. **Parent Chunks (1024 Tokens):** The child chunks are then concatenated in order to create larger "parent chunks" with a 1024-token limit.

This strategy enables a "small-to-large" retrieval pattern: we perform the kNN search against the precise *child chunks*, and once a match is found, we retrieve its associated *parent chunk*. This parent chunk provides the LLM with the full surrounding context (i.e., the "context-aware overlap") needed to formulate a high-quality answer.

4 Embedding & Indexing

4.1 Embedding Model: Multimodal (CLIP)

The project requirements encouraged experimenting with embedding models. Given the highly visual nature of our domain (Astronomy), we made a strategic decision to implement a **multimodal** pipeline from the start.

- **Model Used:** clip-ViT-B-32 from the `sentence-transformers` library.
- **Justification:** CLIP (Contrastive Language-Image Pre-Training) embeds both text and images into a shared 512-dimensional latent space. This is a critical capability for our "Cosmic Compass" agent. It allows a user's text query (e.g., "What is a spiral galaxy?") to be matched against not only text definitions but also *images* of spiral galaxies extracted from the documents. This unified search space is a significant advantage over a text-only system.

As a next step, we will also evaluate a text-specific model like `all-mpnet-base-v2` or a domain-specific (e.g., `SciBERT`) model for text-only queries to compare performance.

4.2 Vector Store: Elasticsearch

We selected **Elasticsearch** as our vector database. The `index_with_es.py` script handles the connection and indexing.

- **Index Creation:** An index (default: `es01-metadata`) is created with a specific mapping to support our needs. The mapping defines a `dense_vector` field with 512 dimensions to match the CLIP model's output.

```

1 mapping = {
2     "properties": {
3         "embedding": {"type": "dense_vector", "dims": 512},
4         "doc_id": {"type": "keyword"},
5         "type": {"type": "keyword"},           // "text" or "image"
6         "chunk_type": {"type": "keyword"}, // "parent", "child", or "n/a"
7         "content": {"type": "text"},
8         "image_path": {"type": "keyword"}
9     }
10 }
11 es_client.indices.create(index=INDEX_NAME, mappings=mapping)

```

Listing 1: Elasticsearch Index Mapping Snippet

4.3 Indexing Pipeline

The indexing process is metadata-driven, ensuring robustness and traceability.

1. The script `index_with_es.py` scans the `METADATA_DIR`.
2. For each JSON manifest, it iterates through the `assets`.

3. **Text Indexing:** It reads the content of each child and parent chunk file, generates an embedding using the CLIP text encoder, and indexes it as a document with `type: "text"`.
4. **Image Indexing:** It opens each image file, generates an embedding using the CLIP image encoder, and indexes it as a document with `type: "image"`.

This process populates our Elasticsearch index with all the content (text and images) and metadata (`doc_id`, `chunk_type`) required for our hierarchical, multimodal RAG system. The included `query_es` function demonstrates a successful k-Nearest Neighbors (kNN) search against this index.

5 Conclusion & Next Steps

We have successfully completed the data ingestion and indexing phase of the project. A robust, metadata-driven pipeline is in place that can process heterogeneous, multimodal documents, perform content-aware hierarchical chunking, and index both text and images into a unified vector space in Elasticsearch.

Our immediate next steps will focus on building the retrieval and generation components on top of this foundation:

- **Implement RAG Logic:** Develop the retrieval logic that performs the "small-to-large" search (search child chunks, retrieve parent chunks).
- **[BONUS] Implement Reranking:** Integrate a reranking model (e.g., BGE Reranker) to improve the relevance of the retrieved parent chunks before they are passed to the LLM.
- **Evaluate Second Embedding Model:** As per the requirements, benchmark the current CLIP model against a text-centric model (like `all-mpnet-base-v2`) for text-only queries.
- **Develop Agent Interface:** Begin building the user-facing application that embodies the "Cosmic Compass" agent, connecting the RAG pipeline to a generative model.